How the Hub Works Now (and What Changed)

As of: 2026-11-03, main branch (commits 8b4b74d / 037e889). Covers the thin-hub pivot (Phases 0–5 of `CittaHub_Revision_Plan.md`; Phase 6 rollout not yet done) plus the post-pivot UI/API hardening done in October–November 2026.

What the Hub is

An internal-only tool (client companies never log in) that pulls invoices from a client's ERP (QuickBooks Online, Odoo, or Excel upload), normalizes them, and submits them to the CittaEFS Gateway, which forwards them to NRS for e-invoicing compliance. Your own team (Admin / Integration Manager / Operator / Auditor) are the only users.

The core change: CittaEFS owns customer/item identity, not the Hub

CittaEFS has no API to register a customer or item — registration only happens through CittaEFS's own Excel bulk-upload templates, in the EFS product itself. The Hub used to not account for this: it ran its own independent `Customer`/`Item` tables as if they were the system of record, with full add/edit/delete forms, unrelated to whatever CittaEFS actually knew about that customer.

Now the Hub tracks a **reference**, not a record: a new `EntityMapping` table stores, per tenant, `sourceErpId` (the code as it exists in the client's ERP) → `cittaReferenceCode` (the ID CittaEFS returns once registered) + a `status` (`PENDING_REGISTRATION` / `MAPPED`). That's it — no address, no email, no business fields owned by the Hub going forward. The old `Customer`/`Item` tables still exist (nothing's been dropped yet — that's Phase 6) but are marked deprecated and no new code writes to them.

The registration gate

Before a **B2B or B2G** invoice reaches the CittaEFS gateway, the worker now checks whether the buyer is `MAPPED` in `EntityMapping`. If not:

- The invoice's status becomes `NEEDS_EFS_REGISTRATION` (a new state, distinct from a validation failure or a transient transmit error).
- An `EntityMapping` row is created/updated as `PENDING_REGISTRATION`.
- The job is pulled out of the active queue — not retried with backoff, because retrying won't register a customer.

B2C and export invoices skip this check entirely — CittaEFS accepts an embedded, unregistered buyer on those, which is how the previous per-invoice submission logic always treated them anyway.

Items are deliberately not gated. CittaEFS's own spec answers conflict on whether items need pre-registration for the API path at all (one sheet says yes, the sheet specifically addressing this says no — full item details just go inline). Gating on unverified, contradictory evidence risked blocking every invoice on a queue that might not need to exist, so it was left out pending a direct answer from CittaEFS.

Clearing the queue: the registration-assist flow

Since there's no API to register a customer, staff now get a guided manual path instead of a dead end:

1. **Customer Registrations tab** (formerly "Customers") lists everyone the Hub has seen, grouped by status — a queue to clear, not a directory to browse or edit. No more add/edit/delete customer forms.
2. **Download EFS Registration File** generates the exact CittaEFS Customer Template (correct sheet name, header row, and column order, pulled from CittaEFS's own spec) pre-filled with every pending customer.
3. Staff upload that file through CittaEFS's own portal — the one manual step that has to happen outside the Hub.
4. **Confirm Registered** — staff paste back the CittaEFS reference code. This flips the mapping to `MAPPED` and automatically resets and resubmits every invoice that was stuck waiting on that customer.

What stayed the same

- ERP adapters (QuickBooks, Odoo), the async queue/worker with exponential backoff and DLQ, RBAC/JWT auth, credential encryption, webhooks, and reconciliation crons — all untouched, all still working (verified via the existing test suite, unchanged).
- The `Invoice` table itself — its schema didn't change, only its documented *intent*: it's now explicitly framed as an internal audit/retry record ("did we submit this, what happened"), not a business ledger a client would ever see.
- The `ItemDictionaryTab` (client-SKU → HS/service-code lookup) — this is a genuinely different thing from customer registration. It's a Hub-owned dictionary used for tax-code inference during ingestion, not a CittaEFS record, so it wasn't touched.
- Credit/debit note handling was already structurally correct (`invoiceTypeCode` → `invoiceType` mapping, billing-reference IRNs) — no changes needed there.

Before vs. after, at a glance

	Before	Now
Customer/item data	Hub-owned `Customer`/`Item` tables, full CRUD UI	CittaEFS-owned; Hub tracks only `EntityMapping` (a reference + status)
Unregistered B2B customer	Invoice submitted anyway, likely rejected by the gateway with no dedicated recovery path	Invoice held in `NEEDS_EFS_REGISTRATION` with a guided export → upload → confirm flow
"Customers" tab	A directory you add/edit/delete profiles in	A registration-status queue you clear
Registering a new customer	No defined path — the Hub had no concept of "registered with CittaEFS"	Download pre-filled EFS template → upload via CittaEFS portal → confirm reference code in the Hub
Invoice type codes (`380`/`381`/`384`)	Sent backwards — credit notes were indistinguishable from ordinary invoices to NRS	Corrected against CittaEFS's own valid-code list

Known gaps (not yet done)

- Staging end-to-end test of the full loop (real ERP → `NEEDS_EFS_REGISTRATION` → export → confirm → resubmit) — needs a connected staging environment.
- Production DB backup before any prod migration — needs ops access.
- Dropping the deprecated `Customer`/`Item` tables — deferred to Phase 6, after a parallel-run period.
- Two exposed secrets found along the way (a hardcoded API key default in `prisma/schema.prisma`, a GitHub token in the git remote URL) still need rotation — that's on you/your GitHub admin, not a code change.

Changes since 2026-09-14

- **UI hardening & six-tab layout** — Overview, Invoices (retry for REJECTED/FAILED), Validation, Directories (Customers/Items server search + pagination, MAPPED/UNMAPPED), Settings (read-only retry backoff), Audit Log. Sidebar fixed to not show partially and closes with ChevronLeft/Right.
- **Gateway rejections are terminal for 4xx** — a 4xx from CittaEFS means the payload is refused. The worker now DLQs immediately, marks the invoice REJECTED and creates a GATEWAY_REJECTED validation error without burning five retries (5xx, 408, 429 still retry).
- **Readable gateway errors** — `src/lib/gatewayErrors.ts` translates CittaEFS's raw SQL / ASP.NET replies (e.g. CK_WM_001_INV_REP_PTR_Type) into plain sentences. The raw text is kept after a "Technical detail" toggle in the Validation tab. Implemented at write time and again on read so existing rows render readably.
- **Normalization positioning clarified** — invoice documents are normalized from QBO/Odoo/Excel into CittaEFS's DTO (canonical schema, invoiceTypeCode mapping, `normalizeCittaCode`). Master data is not normalized into EFS; customers/items remain a reference (EntityMapping) with manual registration.

See `docs/CittaHub_Revision_Plan.md` for the full phase-by-phase breakdown and rationale.